Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

ОТЧЁТ
по лабораторной работе №2
по дисциплине
«Информационные технологии и веб-программирование»
Тема: Работа с базами данных и ORM Sequelize

Выполнила: студентка группы 320604
Хаджинова К.А.

Проверил: Хаджинова Н.В.

Минск 2026

СОДЕРЖАНИЕ

1 ЦЕЛЬ РАБОТЫ 3
2 КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ 3
3 ХОД РАБОТЫ 3
3.1 База данных 3
3.2 Установка и настройка 4
3.3 Модель и миграции 4
3.4 CRUD через ORM 5
3.5 Запуск и проверка 6
3.6 Постраничный вывод и сортировка 9
3.7 Числовое поле revision 10
3.8 Метод PATCH 11
3.9 Связь один-ко-многим и ошибка уникальности 12
3.10 Как вызвать ответ 500 настоящим сбоем 13
3.11 Ответы на контрольные вопросы 16
ЗАКЛЮЧЕНИЕ 17
ПРИЛОЖЕНИЕ А Листинг файла config.js 18
ПРИЛОЖЕНИЕ Б Листинг файла index.js модели 19
ПРИЛОЖЕНИЕ В Листинг файла diagram.js 20
ПРИЛОЖЕНИЕ Г Листинг миграции создания таблицы 21
ПРИЛОЖЕНИЕ Д Листинг миграции добавления поля priority 23
ПРИЛОЖЕНИЕ Е Листинг сида demo-diagrams.js 24
ПРИЛОЖЕНИЕ Ж Листинг файла diagrams.controller.js 25
ПРИЛОЖЕНИЕ И Листинг файла list-query.js 27
ПРИЛОЖЕНИЕ К Листинг миграции добавления поля revision 29
ПРИЛОЖЕНИЕ Л Листинг миграции создания таблицы Projects 30
ПРИЛОЖЕНИЕ М Листинг файла projects.controller.js 32

1 ЦЕЛЬ РАБОТЫ

Освоить подключение реляционной базы данных к Node.js-приложению, использование ORM Sequelize для описания моделей и выполнения CRUD-операций, миграции и сиды. Работа продолжает лабораторную работу 1 для проекта DSLKit, веб-редактора диаграмм как кода, тематика которого соответствует направлению SaaS: коллекция `diagrams` переезжает из массива в памяти в базу данных.

2 КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

ORM. Object-Relational Mapping связывает таблицы реляционной базы с объектами языка. Модель Sequelize описывает таблицу и её колонки, а методы `findAll`, `findByPk`, `create`, `update` и `destroy` превращаются в запросы SQL. Преимущества: код не зависит от диалекта SQL, значения подставляются безопасно, а проверки лежат рядом с моделью. Недостатки: сложные запросы приходится выражать через API библиотеки, а скрытые запросы можно не заметить.

Миграции и сиды. Миграция это файл с двумя функциями: `up` меняет схему, `down` отменяет изменение. Применённые миграции записываются в служебную таблицу, поэтому схема воспроизводится на любом компьютере одной командой и меняется по шагам. Сид заполняет таблицы начальными данными и тоже имеет `up` и `down`.

Диалекты. Sequelize работает с PostgreSQL, SQLite, MySQL и другими базами через драйверы. Для PostgreSQL нужны пакеты `pg` и `pg-hstore`, для SQLite пакет `sqlite3`. Строка подключения `DATABASE_URL` позволяет направить приложение на облачный сервис, например Neon или Supabase, не меняя код.

3 ХОД РАБОТЫ

3.1 База данных

PostgreSQL на компьютере разработчика не установлен, а ставить системное программное обеспечение для учебной работы не требуется, поэтому для локального запуска выбран SQLite: файл базы создаётся сам. Выбор делает файл `config/config.js`. Если задана переменная `DATABASE_URL`, используется PostgreSQL (при `DATABASE_SSL=true` включается TLS для Neon и Supabase с проверкой сертификата сервера, отключить проверку можно только переменной `DATABASE_SSL_REJECT_UNAUTHORIZED=false`), иначе SQLite с файлом из

SQLITE_STORAGE. Миграции, модель и контроллеры от диалекта не зависят. Работа с настоящим PostgreSQL не проверялась: проверены SQLite и сборка настроек для PostgreSQL, а остальное опирается на то, что Sequelize скрывает различия диалектов. Образец переменных лежит в .env.example, сам .env в репозиторий не попадает.

3.2 Установка и настройка

Установлены пакеты sequelize, pg, pg-hstore, sqlite3 и, как зависимость разработки, sequelize-cli. Файл .sequelizerc указывает инструменту каталоги migrations и seeders и файл настроек, поэтому команды npx sequelize-cli работают с теми же параметрами подключения, что и приложение. В package.json добавлены скрипты db:migrate, db:migrate:undo, db:seed, db:seed:undo и db:reset.

3.3 Модель и миграции

Модель Diagram описана в src/models/diagram.js: поля title, notation, sourceCode, priority, owner, числовое revision и внешний ключ projectId, а Sequelize сам добавляет id, createdAt и updatedAt. Допустимые значения notation и priority заданы проверкой isIn в модели и ограничениями CHECK в миграциях, поэтому база не примет значение вне списка, даже если обойти и запрос, и модель. Длины совпадают с типами колонок: title до 80 символов, owner до 60, sourceCode до 10000, а перед сохранением title и owner обрезаются по краям. Идентификатор в адресе не может превышать 2147483647, иначе PostgreSQL ответил бы ошибкой 500. Размер тела запроса ограничен значением 100kb. Имя таблицы Sequelize получает из имени модели во множественном числе, то есть Diagrams.

Схема создаётся только миграциями, метод sync() не используется. Первая миграция создаёт таблицу без поля priority, вторая добавляет его вызовом addColumn со значением по умолчанию normal, а её функция down удаляет колонку. Так показано изменение схемы существующей таблицы, как требует условие. Третья миграция создаёт журнал запросов, четвёртая добавляет числовую колонку revision, пятая создаёт таблицу Projects и колонку projectId. Файлы миграций перечислены в таблице 1.

Таблица 1 – Файлы базы данных

Файл	**Назначение**
`migrations/20261016000001-create-diagrams.js`	Создаёт таблицу Diagrams с колонками id, title, notation, sourceCode, owner, createdAt, updatedAt
`migrations/20261016000002-add-priority-to-diagrams.js`	Добавляет колонку priority, откат удаляет её
`migrations/20261016000004-add-revision-to-diagrams.js`	Добавляет колонку revision с проверкой от 0 до 100000, откат удаляет её
`migrations/20261016000005-create-projects.js`	Создаёт таблицу Projects с уникальным именем и колонку projectId со ссылкой на неё
`seeders/20261016000001-demo-diagrams.js`	Вставляет три демонстрационные диаграммы, откат удаляет их по названию
`src/models/diagram.js`	Модель Sequelize
`src/models/index.js`	Создание подключения и модели по настройкам

3.4 CRUD через ORM

Маршруты, проверка данных и формат ошибок остались как в лабораторной работе 1, изменился только слой контроллеров. Контроллеры получают модель параметром и вызывают методы Sequelize, как показано в таблице 2. Метод `update` возвращает число изменённых строк, поэтому ноль означает ответ 404, после успешной замены запись читается заново и возвращается клиенту. Express 5 передаёт отклонённый Promise в глобальный обработчик ошибок, поэтому конструкции `try/catch` в контроллерах не нужны, а сбой базы даёт ответ 500 без внутренних подробностей.

Таблица 2 – Соответствие маршрутов и методов Sequelize

Маршрут	**Вызов Sequelize**	**Ответ**
`GET /diagrams`	`Diagram.findAll({ order: [['id', 'ASC']] })`	200
`GET /diagrams/:id`	`Diagram.findByPk(id)`	200 или 404
`POST /diagrams`	`Diagram.create(fields)`	201
`PUT /diagrams/:id`	`Diagram.update(fields, { where: { id } })`, затем `findByPk`	200 или 404
`DELETE /diagrams/:id`	`Diagram.destroy({ where: { id } })`	204 или 404

3.5 Запуск и проверка

Порядок запуска: `npm run db:migrate` создаёт таблицу и добавляет колонку `priority`, `npm run db:seed` наполняет базу, `npm run dev` запускает сервер. Проверка на SQLite показала, что список возвращает три записи с полями `createdAt` и `updatedAt`, замена меняет `updatedAt`, первое удаление даёт 204, а повторное 404.

Всего 98 автоматических тестов. Одни проверяют HTTP-маршруты на временной базе SQLite, к которой применены настоящие миграции, включая `PATCH`, числовое поле, проекты и ошибку уникальности. Другие проверяют сами миграции: набор колонок, откат каждой миграции по очереди с сохранением всех ограничений `CHECK`, повторный запуск сида и откат только своих строк, отказ модели и базы от значений вне списков, от отрицательного и огромного числа и от повторного имени проекта. Третьи проверяют настройки подключения, в том числе проверку сертификата по умолчанию. Тесты запускают настоящий `sequelize-cli`, поэтому ошибка в файле миграции будет обнаружена.

Листинг файла config.js приведён в приложении А. Листинг файла index.js модели приведён в приложении Б. Листинг миграции создания таблицы приведён в приложении Г. Листинг миграции добавления поля priority приведён в приложении Д. Листинг сида demo-diagrams.js приведён в приложении Е. Листинг файла diagrams.controller.js приведён в приложении Ж. Листинг миграции добавления поля revision приведён в приложении К. Листинг миграции создания таблицы Projects приведён в приложении Л. Листинг файла projects.controller.js приведён в приложении М.

Миграции и сид выполнены командами `npm run db:migrate` и `npm run db:seed`, их вывод показан на рисунке 1.

Migrations and seed (npm run db:migrate, npm run db:seed)

Console output captured from the processes, shown as text.

```
> sequelize-cli db:migrate
Sequelize CLI [Node: 24.13.1, CLI: 6.6.5, ORM: 6.37.8]
Loaded configuration file "config\config.js".
Using environment "development".
== 20261016000001-create-diagrams: migrating =======
== 20261016000001-create-diagrams: migrated (0.425s)
== 20261016000002-add-priority-to-diagrams: migrating =======
== 20261016000002-add-priority-to-diagrams: migrated (0.086s)
== 20261016000003-create-request-logs: migrating =======
== 20261016000003-create-request-logs: migrated (0.046s)
== 20261016000004-add-revision-to-diagrams: migrating =======
== 20261016000004-add-revision-to-diagrams: migrated (0.086s)
== 20261016000005-create-projects: migrating =======
== 20261016000005-create-projects: migrated (0.111s)

> sequelize-cli db:seed:all
Sequelize CLI [Node: 24.13.1, CLI: 6.6.5, ORM: 6.37.8]
Loaded configuration file "config\config.js".
Using environment "development".
== 20261016000001-demo-diagrams: migrating =======
== 20261016000001-demo-diagrams: migrated (0.089s)
```

Рисунок 1 – Вывод миграций и сида в консоли (вывод процессов, снятый как текст)

Структура созданных таблиц получена запросом к системной таблице `sqlite_master` самой базы и показана на рисунке 2.

Structure of the database

Result of the query SELECT name, sql FROM sqlite_master, shown as text (a query to the real file, not a database viewer window).

```
-- Diagrams
CREATE TABLE "Diagrams" (`id` INTEGER PRIMARY KEY AUTOINCREMENT, `title` VARCHAR(80) NOT NULL, `notation` VARCHAR(10) NOT NULL, `sourceCode` TE
XT NOT NULL, `owner` VARCHAR(60) NOT NULL, `createdAt` DATETIME NOT NULL, `updatedAt` DATETIME NOT NULL, `priority` VARCHAR(10) NOT NULL DEFAUL
T 'normal', `revision` INTEGER NOT NULL DEFAULT 0, `projectId` INTEGER REFERENCES `Projects` (`id`) ON DELETE SET NULL, CONSTRAINT `diagrams_no
tation_check` CHECK (`notation` IN ('UML', 'BPMN', 'ERD', 'DFD', 'IDEF0', 'Petri')), CONSTRAINT `diagrams_title_not_blank` CHECK (length(trim(`
title`)) > 0), CONSTRAINT `diagrams_owner_not_blank` CHECK (length(trim(`owner`)) > 0), CONSTRAINT `diagrams_sourceCode_not_blank` CHECK (lengt
h(trim(`sourceCode`)) > 0), CONSTRAINT `diagrams_title_length` CHECK (length(`title`) <= 80), CONSTRAINT `diagrams_owner_length` CHECK (length
(`owner`) <= 60), CONSTRAINT `diagrams_priority_check` CHECK (`priority` IN ('low', 'normal', 'high')), CONSTRAINT `diagrams_revision_range` CH
ECK ((`revision` >= 0 AND `revision` <= 100000)))

-- Projects
CREATE TABLE "Projects" (`id` INTEGER PRIMARY KEY AUTOINCREMENT, `name` VARCHAR(80) NOT NULL UNIQUE, `createdAt` DATETIME NOT NULL, `updatedAt`
DATETIME NOT NULL, CONSTRAINT `projects_name_not_blank` CHECK (length(trim(`name`)) > 0))

-- RequestLogs
CREATE TABLE `RequestLogs` (`id` INTEGER PRIMARY KEY AUTOINCREMENT, `method` VARCHAR(10) NOT NULL, `path` VARCHAR(200) NOT NULL, `ip` VARCHAR(6
4) NOT NULL, `status` INTEGER, `createdAt` DATETIME NOT NULL, `updatedAt` DATETIME NOT NULL)

-- SequelizeMeta
CREATE TABLE `SequelizeMeta` (`name` VARCHAR(255) NOT NULL UNIQUE PRIMARY KEY)
```

Рисунок 2 – Таблицы Diagrams, Projects, RequestLogs и SequelizeMeta с ограничениями CHECK (результат запроса к файлу базы, снятый как текст)

Запрос `GET /diagrams` открыт в браузере по адресу `http://localhost:3000/diagrams`, три записи из сида показаны на рисунке 3.

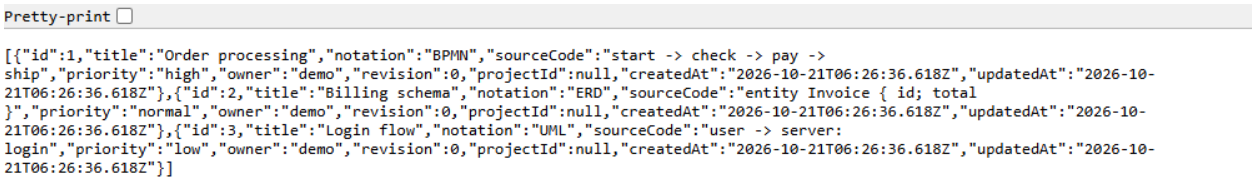


Рисунок 3 – Ответ GET /diagrams в окне браузера: три диаграммы из сида с полями createdAt и updatedAt

Одна запись по адресу `http://localhost:3000/diagrams/1` показана на рисунке 4.

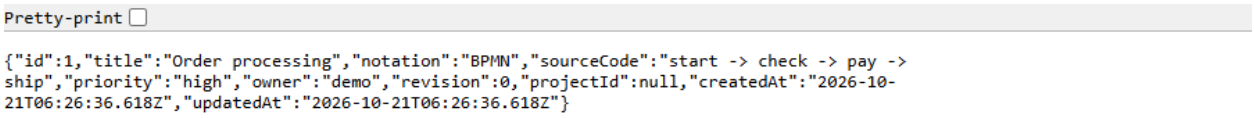


Рисунок 4 – Ответ GET /diagrams/1 в окне браузера: объект одной диаграммы из базы

Запрос несуществующей записи по адресу `http://localhost:3000/diagrams/99` даёт ошибку, показанную на рисунке 5.



Рисунок 5 – Ответ GET /diagrams/99 в окне браузера: JSON с сообщением об отсутствии диаграммы (код 404)

Удаление записи и повторное удаление показаны на рисунке 6. Браузер не отправляет запросы `POST`, `PUT`, `PATCH` и `DELETE` из адресной строки, поэтому такие запросы отправил скрипт `shots/shots.mjs`, а настоящие запросы и ответы сервера выведены на странице.

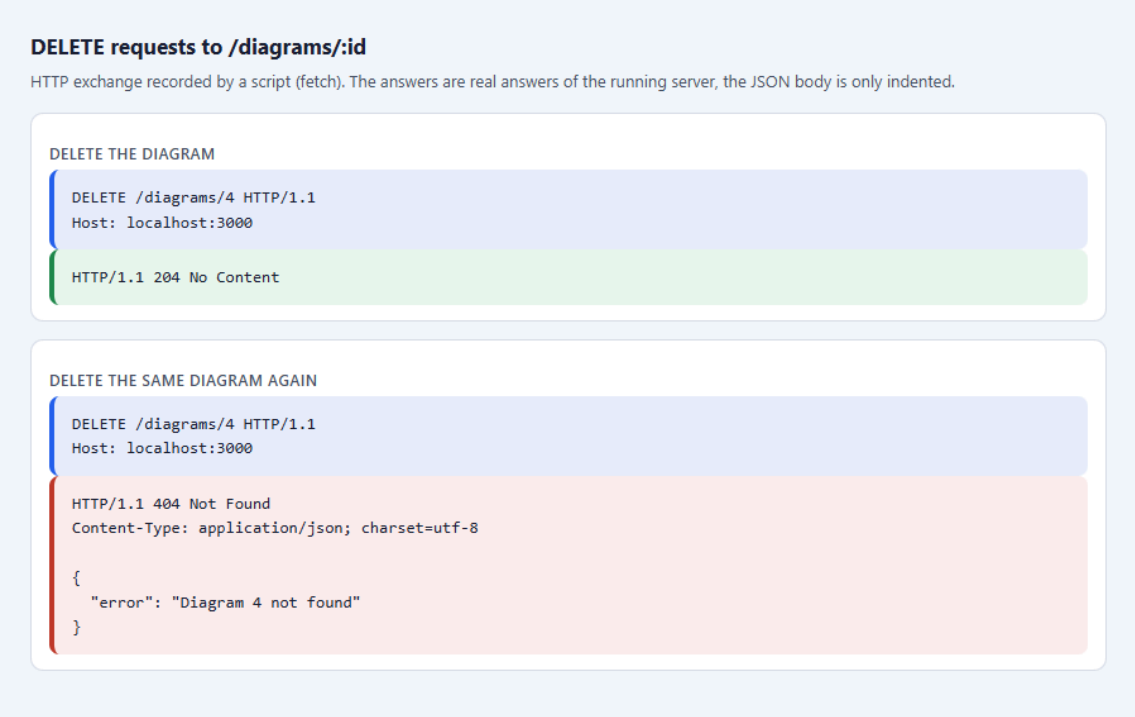


Рисунок 6 – Запросы DELETE: ответ 204 без тела и ответ 404 при повторе, записанные скриптом (не окно Postman)

3.6 Постраничный вывод и сортировка

Список принимает параметры `limit`, `offset`, `sort` и `order`. Параметр `limit` задаёт размер страницы от 1 до 100, `offset` задаёт смещение от 0 до

1000000, `sort` выбирает поле из белого списка, а `order` принимает значения `asc` и `desc`. Без параметров возвращается весь список по возрастанию `id`, поэтому клиенты прежних работ продолжают работать. Ответ остаётся массивом, а полный размер выборки приходит в заголовке `X-Total-Count`. Числа читаются как строки не длиннее девяти цифр, поэтому значения `-1`, `1e9`, `0x10` и огромные числа получают ответ 400 со списком ошибок по полям. Имя поля сортировки проверяется по белому списку и не попадает в запрос к базе без проверки. Равные значения упорядочены по `id`, поэтому страницы не пересекаются. Разбор параметров вынесен в отдельный модуль, он приведён в приложении И.

Запрос `GET /diagrams?limit=2&offset=0&sort=title&order=desc` даёт первую страницу из двух записей, как на рисунке 7.

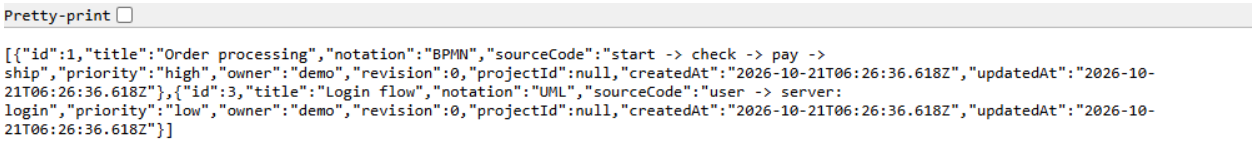


Рисунок 7 – Ответ GET /diagrams с параметрами limit, offset, sort и order в окне браузера

3.7 Числовое поле revision

Поле `revision` хранит номер редакции диаграммы и показывает, как число доходит от базы до ответа. В базе колонка имеет тип `INTEGER` со значением по умолчанию 0 и ограничением `CHECK` от 0 до 100000, в модели то же правило стоит в проверках `isInt`, `min` и `max`, а предел `revisionMax` лежит в `limits.json`. Запрос проверяется до базы: принимается только целое число от 0 до предела. Строка `"5"`, дробь, отрицательное число, `null`, массив и число больше предела получают ответ 400 с полем `revision` в списке ошибок. JSON не умеет хранить `1e999`, он разбирается в бесконечность, и она отклоняется тем же правилом, а значения вроде `1e20` и `9007199254740993` выходят за предел. Без поля в теле значение равно 0. Список можно сортировать по `revision`, числа сравниваются как числа, а не как строки.

Отказ базе и серверу в записи неверных значений, в том числе отрицательной ревизии, показан на рисунке 8.



Рисунок 8 – Ответы 400 со списком проблем полей и при отрицательном revision, записанные скриптом (не окно Postman)

3.8 Метод PATCH

Маршрут `PATCH /diagrams/:id` меняет только присланные поля. Проверка вынесена в `validatePartial`: каждое присланное поле проходит то же правило, что в `POST` и `PUT`, а тело без единого известного поля даёт 400. Контроллер читает запись через `findByPk`, вызывает `diagram.update(fields)` и возвращает обновлённую запись, поэтому проверки модели тоже срабатывают. Метод `PUT` заменяет запись целиком: пропущенные необязательные поля получают значения по умолчанию, а `projectId` сбрасывается в `null`. Метод `PATCH` оставляет пропущенные поля как есть. Поэтому `PUT` идемпотентен, а `PATCH` по определению не обязан быть таким.

Создание, замена и частичное изменение записи в базе показаны на рисунке 9.

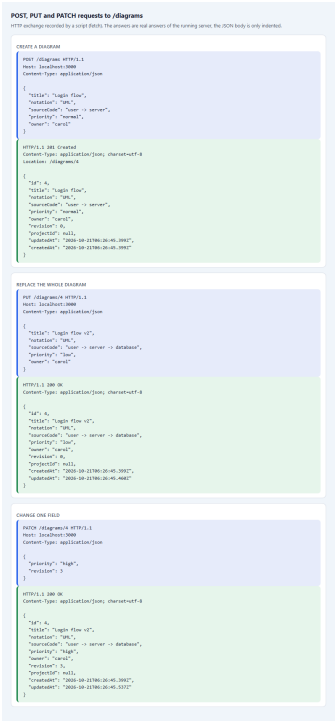


Рисунок 9 – Запросы POST, PUT и PATCH и ответы сервера, записанные скриптом (не окно Postman)

3.9 Связь один-ко-многим и ошибка уникальности

Один проект содержит много диаграмм, а диаграмма относится не более чем к одному проекту. Модель `Project` описана в `src/models/project.js`, связь задана в `src/models/index.js` вызовами `Project.hasMany(Diagram)` и `Diagram.belongsTo(Project)` с внешним ключом `projectId`. В миграции колонка `projectId` ссылается на `Projects.id` с правилом `ON DELETE SET NULL`: при удалении проекта диаграммы остаются, а их `projectId` очищается. Диаграмма может указать только существующий проект, иначе контроллер отвечает 400 с полем `projectId`, а внешний ключ в базе защищает таблицу от записи в обход приложения. Маршруты проекта показаны в таблице 3.

Таблица 3 – Маршруты ресурса projects

Маршрут	**Назначение**	**Ответ**
`GET /projects`	Список проектов с постраничным выводом и сортировкой	200
`POST /projects`	Создание проекта с уникальным именем	201, 400 или 409
`GET /projects/:id`	Один проект	200 или 404

Маршрут	Назначение	Ответ
`GET /projects/:id/diagrams`	Диаграммы проекта с теми же параметрами `limit`, `offset`, `sort`	200 или 404
`DELETE /projects/:id`	Удаление проекта без удаления диаграмм	204 или 404

Имя проекта уникально: колонка `name` создана с ограничением `UNIQUE`. Повторное имя нарушает его, Sequelize выбрасывает `SequelizeUniqueConstraintError`, а глобальный обработчик превращает её в ответ 409 с полем `name` в списке `details`. Проверка только запросом `findOne` перед записью не годится, потому что два одновременных запроса пройдут её оба, поэтому последнее слово остаётся за ограничением базы.

Создание проекта, ответ 409 на повторное имя, перенос диаграммы в проект и выборка диаграмм проекта показаны на рисунке 10.



Рисунок 10 – Запросы к ресурсу projects, включая ответ 409 при повторе имени, записанные скриптом (не окно Postman)

3.10 Как вызвать ответ 500 настоящим сбоем

Ответ 500 вызывается сбоем базы, а не кнопкой. Для SQLite нужно запустить сервер командой `npm start`, затем во втором окне выполнить `npx sequelize-cli db:migrate:undo:all`. Таблицы исчезают под работающим сервером, и первый же запрос, например `GET /diagrams`, не может записать

строку в журнал `RequestLogs`. Клиент получает 500 с телом `{ "error": "Internal server error" }`, а подробности остаются только в консоли сервера. Для PostgreSQL достаточно остановить службу базы или контейнер: запросы дают 500, а `GET /health` отвечает 503. Работу восстанавливает команда `npm run db:migrate` или запуск базы. Тот же сбой воспроизведён в автоматическом тесте удалением таблицы `RequestLogs` при работающем сервере.

Миграции откатаны командой `npx sequelize-cli db:migrate:undo:all` под работающим сервером, и первый же запрос `http://localhost:3000/diagrams` получает ответ 500, как на рисунке 11.



Рисунок 11 – Ответ GET /diagrams после настоящего сбоя базы: JSON с общим сообщением без подробностей (код 500)

Подробности сбоя остаются только в консоли сервера и показаны на рисунке 12.

Console of the server after the real failure

Console output captured from the stdout and stderr of the server process, shown as text (the path of the lab folder is shortened to a dot).

```
> node server.js
Server running on http://localhost:3000
Error
    at Database.<anonymous> (.\node_modules\sequelize\lib\dialects\sqlite\query.js:185:27)
    at .\node_modules\sequelize\lib\dialects\sqlite\query.js:183:50
    at new Promise (<anonymous>)
    at Query.run (.\node_modules\sequelize\lib\dialects\sqlite\query.js:183:12)
    at .\node_modules\sequelize\lib\sequelize.js:315:28
    at process.processTicksAndRejections (node:internal/process/task_queues:103:5)
    at async SQLiteQueryInterface.insert (.\node_modules\sequelize\lib\dialects\abstract\query-interface.js:308:21)
    at async model.save (.\node_modules\sequelize\lib\model.js:2490:35)
    at async RequestLog.create (.\node_modules\sequelize\lib\model.js:1362:12)
    at async .\src\middleware\request-log.js:18:13 {
  name: 'SequelizeDatabaseError',
  parent: [Error: SQLITE_ERROR: no such table: RequestLogs] {
    errno: 1,
    code: 'SQLITE_ERROR',
    sql: 'INSERT INTO `RequestLogs` (`id`,`method`,`path`,`ip`,`createdAt`,`updatedAt`) VALUES (NULL,$1,$2,$3,$4,$5);'
  },
  original: [Error: SQLITE_ERROR: no such table: RequestLogs] {
    errno: 1,
    code: 'SQLITE_ERROR',
    sql: 'INSERT INTO `RequestLogs` (`id`,`method`,`path`,`ip`,`createdAt`,`updatedAt`) VALUES (NULL,$1,$2,$3,$4,$5);'
  },
  sql: 'INSERT INTO `RequestLogs` (`id`,`method`,`path`,`ip`,`createdAt`,`updatedAt`) VALUES (NULL,$1,$2,$3,$4,$5);',
  parameters: {}
}
```

Рисунок 12 – Консоль сервера после сбоя: ошибка отсутствия таблицы RequestLogs (вывод процесса, снятый как текст)

Результат команды `npm test` с числом тестов и покрытием кода показан на рисунке 13.

Result of npm test

Console output captured from node --test, the tail of the output (summary and coverage), exit code 0.

```
> npm test
...
ℹ tests 98
ℹ suites 0
ℹ pass 98
ℹ fail 0
ℹ cancelled 0
ℹ skipped 0
ℹ todo 0
ℹ duration_ms 22231.4997
ℹ start of coverage report
ℹ ---------------------------------------------------------------------------
ℹ file                     | line % | branch % | funcs % | uncovered lines
ℹ ---------------------------------------------------------------------------
ℹ src                      |        |          |         |
ℹ  app.js                  | 100.00 |   100.00 |  100.00 |
ℹ  controllers             |        |          |         |
ℹ   diagrams.controller.js | 100.00 |   100.00 |  100.00 |
ℹ   projects.controller.js | 100.00 |   100.00 |  100.00 |
ℹ  errors.js               | 100.00 |   100.00 |  100.00 |
ℹ  middleware              |        |          |         |
ℹ   error-handler.js       | 100.00 |    94.74 |  100.00 |
ℹ   http.js                | 100.00 |    84.62 |  100.00 |
ℹ   list-query.js          | 100.00 |   100.00 |  100.00 |
ℹ   request-log.js         | 100.00 |    88.89 |   80.00 |
ℹ   validate.js            | 100.00 |   100.00 |  100.00 |
ℹ  models                  |        |          |         |
ℹ   diagram.constants.js   | 100.00 |   100.00 |  100.00 |
ℹ   diagram.js             | 100.00 |   100.00 |  100.00 |
ℹ   index.js               | 100.00 |    50.00 |  100.00 |
ℹ   project.js             | 100.00 |   100.00 |  100.00 |
ℹ   request-log.js         | 100.00 |   100.00 |  100.00 |
ℹ  routes                  |        |          |         |
ℹ   diagrams.routes.js     | 100.00 |   100.00 |  100.00 |
ℹ   projects.routes.js     | 100.00 |   100.00 |  100.00 |
ℹ ---------------------------------------------------------------------------
ℹ all files                | 100.00 |    96.91 |   98.39 |
ℹ ---------------------------------------------------------------------------
ℹ end of coverage report
```

Рисунок 13 – Конец вывода команды npm test: число пройденных тестов и таблица покрытия (вывод процесса, снятый как текст)

3.11 Ответы на контрольные вопросы

5. Как реализовать связь один-ко-многим между моделями? Приведите пример из вашего проекта.

Родительская модель вызывает `hasMany`, дочерняя вызывает `belongsTo`, и обе называют один внешний ключ. В проекте один `Project` имеет много `Diagram`: `Project.hasMany(Diagram, { foreignKey: 'projectId', onDelete: 'SET NULL' })` и `Diagram.belongsTo(Project, { foreignKey: 'projectId' })`. Колонку `projectId` со ссылкой на `Projects.id` создаёт миграция, ассоциация только сообщает о ней Sequelize. После этого доступны методы `project.getDiagrams()` и `diagram.getProject()`, и загрузка одним запросом через `include`. В API это маршрут `GET /projects/:id/diagrams`.

6. Как выполнить пагинацию (limit, offset) в Sequelize?

Метод `findAndCountAll({ limit, offset, order })` возвращает страницу в `rows` и полное число записей в `count`. В работе `count` уходит клиенту в заголовке `X-Total-Count`. Порядок обязателен, иначе страницы могут пересекаться, поэтому равные значения сортируются по `id`. Страница с номером `n` начинается со смещения `(n - 1) * limit`. Размер страницы ограничен числом 100, а `limit` и `offset` читаются как короткие строки цифр, поэтому `-1` и огромные числа дают 400. На очень больших таблицах большое смещение медленно, и тогда выбирают пагинацию по последнему увиденному `id`.

7. Как обработать ошибку уникальности (duplicate entry) при создании записи?

Уникальность задаётся ограничением `UNIQUE` в миграции и `unique: true` в модели. Нарушение приводит к исключению `SequelizeUniqueConstraintError`, у которого в `errors[0].path` лежит имя поля. Его ловят в обработчике и отвечают 409 Conflict, а не 500, потому что виновата не база, а повторное значение. В работе так отвечает `POST /projects` с уже занятым именем. Предварительный `findOne` не заменяет ограничение: между проверкой и записью может вклиниться другой запрос.

ЗАКЛЮЧЕНИЕ

Для проекта DSLKit коллекция `diagrams` из лабораторной работы 1 переведена с массива на базу данных через Sequelize: модели `Diagram` и `Project` со связью один-ко-многим, пять миграций для таблицы, поля `priority`, журнала запросов, числового поля `revision` и проектов, сид с тремя диаграммами и контроллеры на методах ORM. Добавлены метод `PATCH`, ответ 409 при повторном имени проекта и проверка числа от 0 до предела. Локально работа использует SQLite, а переменная `DATABASE_URL` переключает её на PostgreSQL или облачный Neon без изменения кода. 98 автоматических тестов подтверждают маршруты, ограничения, миграции с откатом и сид. Запуск на настоящем PostgreSQL не проверялся, структура базы и запросы подтверждены снимками браузера и записью обмена, снятой скриптом.

ПРИЛОЖЕНИЕ А
(обязательное)
Листинг файла config.js

```
const { readFileSync } = require('node:fs');

try {
  process.loadEnvFile();
} catch {
  // .env is optional, defaults apply
}

// DATABASE_URL selects PostgreSQL, otherwise a local SQLite fil
e is used.
// With DATABASE_SSL=true the server certificate is verified; on
ly DATABASE_SSL_REJECT_UNAUTHORIZED=false turns that off.
// DATABASE_SSL_CA is a path to a PEM file with the certificate
authority, the safe way to trust a private or self-signed serve
r.
const fromEnv = () => {
  const { DATABASE_URL, DATABASE_SSL, DATABASE_SSL_REJECT_UNAUTH
ORIZED, DATABASE_SSL_CA, SQLITE_STORAGE, NODE_ENV } = process.en
v;
  if (!DATABASE_URL) return { dialect: 'sqlite', storage: SQLITE
_STORAGE || './dev.sqlite', logging: false };
  if (NODE_ENV === 'production' && (DATABASE_SSL !== 'true' || D
ATABASE_SSL_REJECT_UNAUTHORIZED === 'false')) {
    throw new Error('In production DATABASE_SSL=true is required
and certificate checks cannot be turned off, use DATABASE_SSL_CA
instead');
  }
  return {
    use_env_variable: 'DATABASE_URL',
    dialect: 'postgres',
    logging: false,
    ...(DATABASE_SSL === 'true' && { dialectOptions: { ssl: { re
quire: true, rejectUnauthorized: DATABASE_SSL_REJECT_UNAUTHORIZE
D !== 'false', ...(DATABASE_SSL_CA && { ca: readFileSync(DATABAS
E_SSL_CA, 'utf8') }) } } }),
  };
};

// One config for every environment: tests override SQLITE_STORA
GE.
module.exports = { development: fromEnv(), test: fromEnv(), prod
uction: fromEnv() };
```

ПРИЛОЖЕНИЕ Б

(обязательное)

Листинг файла index.js модели

```
const { Sequelize } = require('sequelize');
const { defineDiagram } = require('./diagram');
const { defineRequestLog } = require('./request-log');
const { defineProject } = require('./project');
const allConfigs = require('../../config/config');

// Builds the connection from config/config.js; NODE_ENV picks t
he block.
const createModels = (config = allConfigs[process.env.NODE_ENV |
| 'development']) => {
  const { use_env_variable: urlVariable, ...options } = config;
  const sequelize = urlVariable ? new Sequelize(process.env[urlV
ariable], options) : new Sequelize(options);
  const Diagram = defineDiagram(sequelize);
  const Project = defineProject(sequelize);
  // One-to-many: a project has many diagrams, a diagram belongs
to one project. Deleting a project clears projectId of its diagr
ams.
  Project.hasMany(Diagram, { foreignKey: 'projectId', as: 'diagr
ams', onDelete: 'SET NULL' });
  Diagram.belongsTo(Project, { foreignKey: 'projectId', as: 'pro
ject' });
  return { sequelize, Diagram, Project, RequestLog: defineReques
tLog(sequelize) };
};

module.exports = { createModels };
```

ПРИЛОЖЕНИЕ В

(обязательное)

Листинг файла diagram.js

```
const { DataTypes } = require('sequelize');
const { NOTATIONS, PRIORITIES, DEFAULT_PRIORITY, LIMITS } = requ
ire('./diagram.constants');

// The table is created by migrations, never by sequelize.sync
().
const defineDiagram = (sequelize) =>
  sequelize.define('Diagram', {
    title: { type: DataTypes.STRING(LIMITS.title), allowNull: fa
lse, validate: { len: [1, LIMITS.title] } },
    notation: { type: DataTypes.STRING(10), allowNull: false, va
lidate: { isIn: [NOTATIONS] } },
    sourceCode: { type: DataTypes.TEXT, allowNull: false, valida
te: { len: [1, LIMITS.sourceCode] } },
    priority: { type: DataTypes.STRING(10), allowNull: false, de
faultValue: DEFAULT_PRIORITY, validate: { isIn: [PRIORITIES] }
},
    owner: { type: DataTypes.STRING(LIMITS.owner), allowNull: fa
lse, validate: { len: [1, LIMITS.owner] } },
    // A counter that is never negative, the same range as the C
HECK in the migration.
    revision: { type: DataTypes.INTEGER, allowNull: false, defau
ltValue: 0, validate: { isInt: true, min: 0, max: LIMITS.revisio
nMax } },
    // Foreign key of the one-to-many link, see the associations
in index.js.
    projectId: { type: DataTypes.INTEGER, allowNull: true },
  });

module.exports = { defineDiagram };
```

ПРИЛОЖЕНИЕ Г

(обязательное)

Листинг миграции создания таблицы

```
// A migration is a snapshot, so the allowed values and limits a
re copied here instead of imported.
const NOTATIONS = ['UML', 'BPMN', 'ERD', 'DFD', 'IDEF0', 'Petr
i'];
const LIMITS = { title: 80, owner: 60 };

module.exports = {
  async up(queryInterface, Sequelize) {
    const { fn, col, where, Op } = Sequelize;
    await queryInterface.createTable('Diagrams', {
      id: { allowNull: false, autoIncrement: true, primaryKey: t
rue, type: Sequelize.INTEGER },
      title: { allowNull: false, type: Sequelize.STRING(LIMITS.t
itle) },
      notation: { allowNull: false, type: Sequelize.STRING(10)
},
      sourceCode: { allowNull: false, type: Sequelize.TEXT },
      owner: { allowNull: false, type: Sequelize.STRING(LIMITS.o
wner) },
      createdAt: { allowNull: false, type: Sequelize.DATE },
      updatedAt: { allowNull: false, type: Sequelize.DATE },
    });
    await queryInterface.addConstraint('Diagrams', {
      type: 'check',
      fields: ['notation'],
      name: 'diagrams_notation_check',
      where: { notation: { [Op.in]: NOTATIONS } },
    });
    // The API trims and checks every text, these checks keep th
e table clean for any other writer (SQLite does not enforce VARC
HAR length).
    for (const column of ['title', 'owner', 'sourceCode']) {
      await queryInterface.addConstraint('Diagrams', {
        type: 'check',
        fields: [column],
        name: `diagrams_${column}_not_blank`,
        where: where(fn('length', fn('trim', col(column))), { [O
p.gt]: 0 }),
      });
    }
    for (const column of ['title', 'owner']) {
      await queryInterface.addConstraint('Diagrams', {
        type: 'check',
        fields: [column],
        name: `diagrams_${column}_length`,
        where: where(fn('length', col(column)), { [Op.lte]: LIMI
```

```
TS[column] }),
      });
    }
  },

  async down(queryInterface) {
    await queryInterface.dropTable('Diagrams');
  },
};
```

ПРИЛОЖЕНИЕ Д

(обязательное)

Листинг миграции добавления поля priority

```
const PRIORITIES = ['low', 'normal', 'high'];

module.exports = {
  async up(queryInterface, Sequelize) {
    await queryInterface.addColumn('Diagrams', 'priority', {
      allowNull: false,
      type: Sequelize.STRING(10),
      defaultValue: 'normal',
    });
    await queryInterface.addConstraint('Diagrams', {
      type: 'check',
      fields: ['priority'],
      name: 'diagrams_priority_check',
      where: { priority: { [Sequelize.Op.in]: PRIORITIES } },
    });
  },

  async down(queryInterface) {
    await queryInterface.removeConstraint('Diagrams', 'diagrams_
priority_check');
    if (queryInterface.sequelize.getDialect() === 'sqlite') {
      // The generic removeColumn rebuilds a SQLite table from i
ts columns only and loses every CHECK,
      // while the native DROP COLUMN (SQLite 3.35+) leaves the
rest of the table as it was.
      await queryInterface.sequelize.query('ALTER TABLE `Diagram
s` DROP COLUMN `priority`');
    } else {
      await queryInterface.removeColumn('Diagrams', 'priority');
    }
  },
};
```

ПРИЛОЖЕНИЕ Е
(обязательное)
Листинг сида demo-diagrams.js

```
const { ROWS } = require('../seed-data/diagrams');

// Every seeded row has this owner, so the seed can tell its row
s from the ones users created.
const MARKER = 'demo';

// Idempotent per row: a row that is already there is left alon
e, a deleted one comes back, nothing is duplicated.
module.exports = {
  async up(queryInterface) {
    const now = new Date();
    for (const row of ROWS) {
      const exists = await queryInterface.rawSelect('Diagrams',
{ where: { owner: MARKER, title: row.title } }, ['id']);
      if (!exists) await queryInterface.bulkInsert('Diagrams',
[{ ...row, owner: MARKER, createdAt: now, updatedAt: now }]);
    }
  },

  async down(queryInterface, Sequelize) {
    await queryInterface.bulkDelete('Diagrams', { owner: MARKER,
title: { [Sequelize.Op.in]: ROWS.map((row) => row.title) } });
  },
};
```

ПРИЛОЖЕНИЕ Ж
(обязательное)
Листинг файла diagrams.controller.js

```
const { notFound, badRequest } = require('../errors');
const { parseListQuery } = require('../middleware/list-query');

const missing = (id) => notFound(`Diagram ${id} not found`);

const SORTABLE = ['id', 'title', 'notation', 'priority', 'owne
r', 'revision', 'createdAt'];

// Express 5 forwards a rejected promise to the error handler, s
o no try/catch is needed here.
const createDiagramsController = ({ Diagram, Project }) => {
  // The foreign key is also guarded by the database, but a clea
r 400 beats a database error for a project that does not exist.
  const ensureProject = async (projectId) => {
    if (projectId === undefined || projectId === null || (await
Project.findByPk(projectId))) return;
    throw badRequest('Invalid diagram data', [{ field: 'projectI
d', message: `project ${projectId} does not exist` }]);
  };

  return {
    // ?limit=&offset=&sort=&order= (see list-query.js). The ans
wer stays an array, the full size comes in X-Total-Count.
    list: async (req, res) => {
      const { limit, offset, sort, order } = parseListQuery(req.
query, SORTABLE);
      const { count, rows } = await Diagram.findAndCountAll({
        order: [
          [sort, order.toUpperCase()],
          ['id', 'ASC'],
        ],
        limit,
        offset,
      });
      res.set('X-Total-Count', String(count));
      res.json(rows);
    },

    get: async (req, res, next) => {
      const diagram = await Diagram.findByPk(req.diagramId);
      return diagram ? res.json(diagram) : next(missing(req.diag
ramId));
    },

    create: async (req, res) => {
      await ensureProject(req.diagramFields.projectId);
```

```
      const diagram = await Diagram.create(req.diagramFields);
      res.status(201).location(`/diagrams/${diagram.id}`).json(d
iagram);
    },

    replace: async (req, res, next) => {
      await ensureProject(req.diagramFields.projectId);
      const [updated] = await Diagram.update(req.diagramFields,
{ where: { id: req.diagramId } });
      return updated ? res.json(await Diagram.findByPk(req.diagr
amId)) : next(missing(req.diagramId));
    },

    // PATCH: only the sent fields change, the model validators
still run on them.
    update: async (req, res, next) => {
      const diagram = await Diagram.findByPk(req.diagramId);
      if (!diagram) return next(missing(req.diagramId));
      await ensureProject(req.diagramFields.projectId);
      await diagram.update(req.diagramFields);
      return res.json(diagram);
    },

    remove: async (req, res, next) => {
      const removed = await Diagram.destroy({ where: { id: req.d
iagramId } });
      return removed ? res.status(204).end() : next(missing(req.
diagramId));
    },
  };
};

module.exports = { createDiagramsController };
```

ПРИЛОЖЕНИЕ И
(обязательное)
Листинг файла list-query.js

```
const { badRequest } = require('../errors');

// Limits of the list query. The page size is capped so one requ
est cannot ask for the whole table,
// and the numbers are read as short digit strings, so 1e9, -1,
0x10 or a huge value are refused with 400.
const MAX_LIMIT = 100;
const MAX_OFFSET = 1000000;
const ORDERS = ['asc', 'desc'];

// Reads ?limit=, ?offset=, ?sort= and ?order= from req.query.
// sortable: the field names a client may sort by (a whitelist,
so a field name never reaches the database unchecked).
// Without limit the whole list is returned (the clients of the
earlier labs expect that), without sort the order is by id.
// A bad value throws the 400 error with the list of problems, E
xpress 5 passes it to the error handler.
const parseListQuery = (query, sortable) => {
  const problems = [];
  const integer = (name, min, max) => {
    const value = query[name];
    if (value === undefined) return undefined;
    if (typeof value !== 'string' || !/^\d{1,9}$/.test(value) ||
Number(value) < min || Number(value) > max) {
      problems.push({ field: name, message: `${name} must be an
integer from ${min} to ${max}` });
      return undefined;
    }
    return Number(value);
  };
  const limit = integer('limit', 1, MAX_LIMIT);
  const offset = integer('offset', 0, MAX_OFFSET) ?? 0;

  const sort = query.sort ?? 'id';
  if (typeof sort !== 'string' || !sortable.includes(sort)) {
    problems.push({ field: 'sort', message: `sort must be one o
f: ${sortable.join(', ')}` });
  }
  const order = typeof query.order === 'string' ? query.order.to
LowerCase() : (query.order ?? 'asc');
  if (!ORDERS.includes(order)) problems.push({ field: 'order', m
essage: 'order must be asc or desc' });

  if (problems.length) throw badRequest('Invalid list query', pr
oblems);
  return { limit, offset, sort, order };
```

```
};

module.exports = { parseListQuery, MAX_LIMIT, MAX_OFFSET };
```

ПРИЛОЖЕНИЕ К

(обязательное)

Листинг миграции добавления поля revision

```
// A migration is a snapshot, so the limit is copied here instea
d of imported.
const REVISION_MAX = 100000;

// A numeric column that is never negative: the CHECK keeps it c
lean for any writer, not only for the API.
module.exports = {
  async up(queryInterface, Sequelize) {
    await queryInterface.addColumn('Diagrams', 'revision', {
      allowNull: false,
      type: Sequelize.INTEGER,
      defaultValue: 0,
    });
    await queryInterface.addConstraint('Diagrams', {
      type: 'check',
      fields: ['revision'],
      name: 'diagrams_revision_range',
      where: { revision: { [Sequelize.Op.gte]: 0, [Sequelize.Op.
lte]: REVISION_MAX } },
    });
  },

  async down(queryInterface) {
    await queryInterface.removeConstraint('Diagrams', 'diagrams_
revision_range');
    if (queryInterface.sequelize.getDialect() === 'sqlite') {
      // The generic removeColumn rebuilds a SQLite table from i
ts columns only and loses every CHECK,
      // while the native DROP COLUMN (SQLite 3.35+) leaves the
rest of the table as it was.
      await queryInterface.sequelize.query('ALTER TABLE `Diagram
s` DROP COLUMN `revision`');
    } else {
      await queryInterface.removeColumn('Diagrams', 'revision');
    }
  },
};
```

ПРИЛОЖЕНИЕ Л

(обязательное)

Листинг миграции создания таблицы Projects

```
// A migration is a snapshot, so the limit is copied here instea
d of imported.
const NAME_MAX = 80;

// One project has many diagrams (one-to-many). The name is uniq
ue, a second project with the same name is a database error
// that the API turns into 409. Deleting a project keeps its dia
grams and clears their projectId.
module.exports = {
  async up(queryInterface, Sequelize) {
    const { fn, col, where, Op } = Sequelize;
    await queryInterface.createTable('Projects', {
      id: { allowNull: false, autoIncrement: true, primaryKey: t
rue, type: Sequelize.INTEGER },
      name: { allowNull: false, unique: true, type: Sequelize.ST
RING(NAME_MAX) },
      createdAt: { allowNull: false, type: Sequelize.DATE },
      updatedAt: { allowNull: false, type: Sequelize.DATE },
    });
    await queryInterface.addConstraint('Projects', {
      type: 'check',
      fields: ['name'],
      name: 'projects_name_not_blank',
      where: where(fn('length', fn('trim', col('name'))), { [Op.
gt]: 0 }),
    });
    await queryInterface.addColumn('Diagrams', 'projectId', {
      allowNull: true,
      type: Sequelize.INTEGER,
      references: { model: 'Projects', key: 'id' },
      onDelete: 'SET NULL',
    });
    await queryInterface.addIndex('Diagrams', ['projectId'], { n
ame: 'diagrams_project_id' });
  },

  async down(queryInterface) {
    await queryInterface.removeIndex('Diagrams', 'diagrams_proje
ct_id');
    if (queryInterface.sequelize.getDialect() === 'sqlite') {
      // The generic removeColumn would rebuild the table and lo
se every CHECK, see the priority migration.
      await queryInterface.sequelize.query('ALTER TABLE `Diagram
s` DROP COLUMN `projectId`');
    } else {
      await queryInterface.removeColumn('Diagrams', 'projectI
```

```
d');
    }
    await queryInterface.dropTable('Projects');
  },
};
```

ПРИЛОЖЕНИЕ М
(обязательное)
Листинг файла projects.controller.js

```
const { notFound } = require('../errors');
const { parseListQuery } = require('../middleware/list-query');

const PROJECT_SORTABLE = ['id', 'name', 'createdAt'];
const DIAGRAM_SORTABLE = ['id', 'title', 'notation', 'priority',
'owner', 'revision', 'createdAt'];

const missing = (id) => notFound(`Project ${id} not found`);

// A project groups diagrams. Creating a project with a name tha
t already exists breaks the UNIQUE constraint,
// Sequelize throws SequelizeUniqueConstraintError and the error
handler answers 409.
const createProjectsController = ({ Project, Diagram }) => ({
  list: async (req, res) => {
    const { limit, offset, sort, order } = parseListQuery(req.qu
ery, PROJECT_SORTABLE);
    const { count, rows } = await Project.findAndCountAll({
      order: [
        [sort, order.toUpperCase()],
        ['id', 'ASC'],
      ],
      limit,
      offset,
    });
    res.set('X-Total-Count', String(count));
    res.json(rows);
  },

  get: async (req, res, next) => {
    const project = await Project.findByPk(req.projectId);
    return project ? res.json(project) : next(missing(req.projec
tId));
  },

  // The many side of the one-to-many link: all diagrams of one
project, with the same paging as /diagrams.
  listDiagrams: async (req, res, next) => {
    const project = await Project.findByPk(req.projectId);
    if (!project) return next(missing(req.projectId));
    const { limit, offset, sort, order } = parseListQuery(req.qu
ery, DIAGRAM_SORTABLE);
    const { count, rows } = await Diagram.findAndCountAll({
      where: { projectId: project.id },
      order: [
        [sort, order.toUpperCase()],
```

```
        ['id', 'ASC'],
      ],
      limit,
      offset,
    });
    res.set('X-Total-Count', String(count));
    return res.json(rows);
  },

  create: async (req, res) => {
    const project = await Project.create(req.projectFields);
    res.status(201).location(`/projects/${project.id}`).json(pro
ject);
  },

  // The diagrams of the project stay, the database clears their
projectId (ON DELETE SET NULL).
  remove: async (req, res, next) => {
    const removed = await Project.destroy({ where: { id: req.pro
jectId } });
    return removed ? res.status(204).end() : next(missing(req.pr
ojectId));
  },
});

module.exports = { createProjectsController };
```